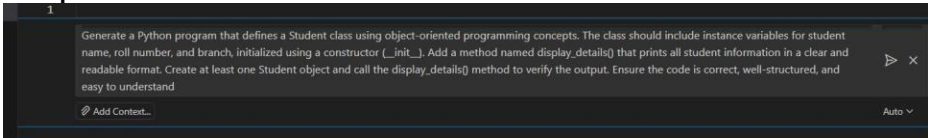


2303A51832

B_26

AssignmentNumber:6.3(Present assignment number)/24(Total number of assignments)		
Q.No.	Question	Expected Time to complete
1	Lab 6: AI-Based Code Completion – Classes, Loops, and Conditionals Lab Objectives • To explore AI-powered auto-completion features for core Python constructs such as classes, loops, and conditional statements. • To analyze how AI tools suggest logic for object-oriented programming and control structures. • To evaluate the correctness, readability, and completeness of AI-generated Python code. Lab Outcomes (LOs) After completing this lab, students will be able to: • Use AI tools to generate and complete Python class definitions and methods. • Understand and assess AI-suggested loop constructs for iterative tasks. • Generate and evaluate conditional statements using AI-driven prompts. • Critically analyze AI-assisted code for correctness, clarity, and efficiency. Task Description #1: Classes (Student Class) Scenario You are developing a simple student information management module. Task • Use an AI tool (GitHub Copilot / Cursor AI / Gemini) to complete a Student class. • The class should include attributes such as name, roll number, and branch. • Add a method <code>display_details()</code> to print student information. • Execute the code and verify the output. • Analyze the code generated by the AI tool for correctness and clarity. Expected Output #1 • A Python class with a constructor (<code>__init__</code>) and a <code>display_details()</code> method. • Sample object creation and output displayed on the console. • Brief analysis of AI-generated code. Prompt:  Code:	Week3 - Wednesday

```
AI assignment_6.3.py X
AI assignment_6.3.py > ...
1 class Student:
2     """A class to represent a student."""
3
4     def __init__(self, name, roll_number, branch):
5         """
6         Initialize student with name, roll number, and branch.
7
8         Args:
9             name (str): Student's name
10            roll_number (int): Student's roll number
11            branch (str): Student's branch of study
12        """
13        self.name = name
14        self.roll_number = roll_number
15        self.branch = branch
16
17    def display_details(self):
18        """Display all student information in a readable format."""
19        print("-" * 40)
20        print("Student Information")
21        print("-" * 40)
22        print(f"Name: {self.name}")
23        print(f"Roll Number: {self.roll_number}")
24        print(f"Branch: {self.branch}")
25        print("-" * 40)
26
27    # Create Student objects and display their details
28    student1 = Student("Alice Johnson", 101, "Computer Science")
29    student1.display_details()
30
31    student2 = Student("Bob Smith", 102, "Electronics")
32    student2.display_details()
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\abbua\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/abbua/AppData/Local/Microsoft/WindowsApps/python.11.exe "C:/Users/abbua/OneDrive/Desktop/AI ASSISTED CODING/AI assignment_6.3.py"
Student Information
-----
Name: Alice Johnson
Roll Number: 101
Branch: Computer Science
-----
Student Information
-----
Name: Bob Smith
Roll Number: 102
Branch: Electronics
-----
PS C:\Users\abbua\OneDrive\Desktop\AI ASSISTED CODING>
```

Explanation:

This code defines a Student class used to store and display basic student information. The class has a constructor (`__init__`) that initializes three attributes: name, roll number, and branch when a student object is created. The `display_details()` method neatly prints the student's information with clear labels and separators, making the output easy to read. At the end of the program, two Student objects are created with different details, and the `display_details()` method is called for each object to show their information on the console. Overall, the code is simple, well-structured, and clearly demonstrates how classes and objects work in Python.

Task Description #2: Loops (Multiples of a Number)

Scenario

You are writing a utility function to display multiples of a given number.

Task

- Prompt the AI tool to generate a function that prints the first 10 multiples of a given number using a loop.
- Analyze the generated loop logic.
- Ask the AI to generate the same functionality using another controlled looping structure (e.g., while instead of for).

Expected Output #2

- Correct loop-based Python implementation.
- Output showing the first 10 multiples of a number.
- Comparison and analysis of different looping approaches.

Prompt:

Generate a Python function that accepts an integer as input and prints the first 10 multiples of that number using a for loop. Ensure the loop logic is correct and the output is properly formatted. Explain how the loop iterates to generate the multiples. Then generate another Python function that performs the same task using a while loop instead of a for loop. Finally, compare both looping approaches in terms of readability, control, and suitability for this task.

Add Context...

Auto

Code:

AI assignment_6.3.py > print_multiples_for

```
1 def print_multiples_for(n):
2     """Prints the first 10 multiples of the given integer using a for loop."""
3     for i in range(1, 11):
4         print(f"{n} x {i} = {n * i}")
5
6 def print_multiples_while(n):
7     """Prints the first 10 multiples of the given integer using a while loop."""
8     i = 1
9     while i <= 10:
10        print(f"{n} x {i} = {n * i}")
11        i += 1
12
13 # Example usage
14 number = int(input("Enter an integer: "))
15 print_multiples_for(number)
16 print_multiples_while(number)
17
18
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python + - W ... |

```
PS C:\Users\abbua\OneDrive\Desktop\AI_ASSISTED CODING> & C:\Users\abbua\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:\Users\abbua\OneDrive\Desktop\AI_ASSISTED CODING\AI_assignment_6.3.py"
Enter an integer: 5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
PS C:\Users\abbua\OneDrive\Desktop\AI_ASSISTED CODING>
```

Explanation:

This code defines two functions to print the first 10 multiples of a given number using different looping structures. The `print_multiples_for()` function uses a for loop that runs from 1 to 10 and prints the multiplication result for each value. The `print_multiples_while()` function performs the same task using a while loop, where a counter variable starts at 1 and increments until it reaches 10. The program then asks the user to enter an integer and calls both functions, showing how the same logic can be implemented using both for and while loops in Python.

Task Description #3: Conditional Statements (Age Classification)

Scenario

You are building a basic classification system based on age.

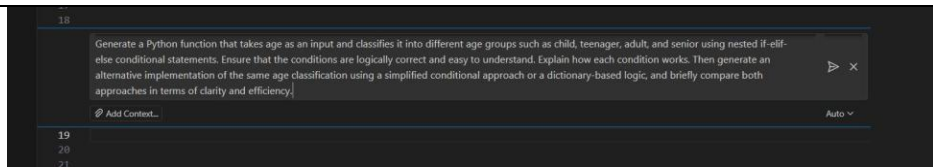
Task

- Ask the AI tool to generate nested if-elif-else conditional statements to classify age groups (e.g., child, teenager, adult, senior).
- Analyze the generated conditions and logic.
- Ask the AI to generate the same classification using alternative conditional structures (e.g., simplified conditions or dictionary-based logic).

Expected Output #3

- A Python function that classifies age into appropriate groups.
- Clear and correct conditional logic.
- Explanation of how the conditions work.

Prompt:



Code:

```
[1] 0a # Approach 1: Nested if-elif-else conditional statements
def classify_age_nested(age):
    """
    Classifies age into groups using nested if-elif-else statements.
    - Child: 0-12 years
    - Teenager: 13-19 years
    - Adult: 20-64 years
    - Senior: 65+ years
    """
    if age < 0:
        return "Invalid age"
    elif age <= 12:
        return "Child"
    elif age <= 19:
        return "Teenager"
    elif age <= 64:
        return "Adult"
    else:
        return "Senior"

# Approach 2: Dictionary-based logic
def classify_age_dict(age):
    """
    Classifies age using a dictionary with tuples as keys.
    More concise and easier to modify age ranges.
    """
    age_groups = [
        (0, 12, "Child"),
        (13, 19, "Teenager"),
        (20, 64, "Adult"),
        (65, 150, "Senior")
    ]
```

```
[1] 0a ]
    for min_age, max_age, group in age_groups:
        if min_age <= age <= max_age:
            return group
    return "Invalid age"

# Example usage
if __name__ == "__main__":
    test_ages = [5, 15, 30, 70, -1]

    print("\nNested if-elif-else approach:")
    for age in test_ages:
        print(f"Age {age}: {classify_age_nested(age)}")

    print("\nDictionary-based approach:")
    for age in test_ages:
        print(f"Age {age}: {classify_age_dict(age)}")

--- Nested if-elif-else approach:
Age 5: Child
Age 15: Teenager
Age 30: Adult
Age 70: Senior
Age -1: Invalid age

Dictionary-based approach:
Age 5: Child
Age 15: Teenager
Age 30: Adult
Age 70: Senior
Age -1: Invalid age
```

Explanation:

This code shows two different ways to classify a person's age into groups such as child, teenager, adult, and senior. In the first approach, the `classify_age_nested()` function uses if-elif-else statements to check the age step by step and return the appropriate category, while also handling invalid negative ages. In the second approach, the `classify_age_dict()` function stores age ranges and their labels in a list of tuples and uses a loop to find where the given age fits, making the logic more compact and easier to modify. In the example usage section, a list of test ages is checked using both functions, and the results are printed to compare how each approach works.

Task Description #4: For and While Loops (Sum of First n Numbers)

Scenario

You need to calculate the sum of the first n natural numbers.

Task

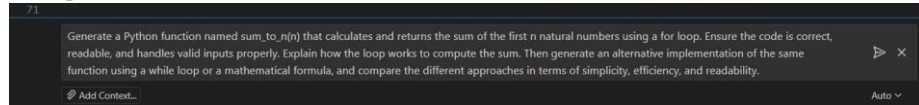
- Use AI assistance to generate a `sum_to_n()` function using a for loop.

- Analyze the generated code.
- Ask the AI to suggest an alternative implementation using a while loop or a mathematical formula.

Expected Output #4

- Python function to compute the sum of first n numbers.
- Correct output for sample inputs.
- Explanation and comparison of different approaches.

Prompt:



Code:

```

C:\> Users\abbas > aaapy > ...
1 # Approach 1: Using a for loop
2 def sum_to_n(n):
3     """Calculate sum of first n natural numbers using a for loop."""
4     total = 0
5     for i in range(1, n + 1):
6         total += i
7     return total
8
9 # Test cases
10 print(sum_to_n(5)) # Output: 15
11 print(sum_to_n(10)) # Output: 55
12 print(sum_to_n(100)) # Output: 5050
13
14
15 # Approach 2: Using a while loop
16 def sum_to_n_while(n):
17     """Calculate sum of first n natural numbers using a while loop."""
18     total = 0
19     i = 1
20     while i <= n:
21         total += i
22         i += 1
23     return total
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
```

Explanation:

This code shows **three different ways** to calculate the sum of the first n natural numbers. The first function uses a **for loop**, where numbers from 1 to n are added one by one to a total variable. The second function uses a **while loop**, which repeats the addition process until the counter reaches n . The third function uses a **mathematical formula** $n(n+1)/2$, which gives the result instantly without looping, making it the fastest method. Test cases check whether the functions give correct outputs, and the final assertion confirms that all three approaches produce the same result, proving their correctness.

Task Description #5: Classes (Bank Account Class)

Scenario

You are designing a basic banking application.

Task

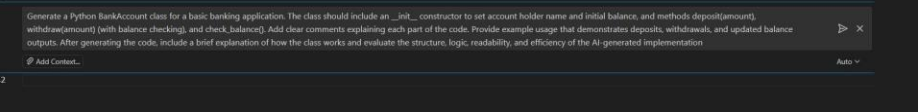
- Use AI tools to generate a Bank Account class with methods such as deposit(), withdraw(), and check_balance().

- Analyze the AI-generated class structure and logic.
- Add meaningful comments and explain the working of the code.

Expected Output #5

- Complete Python Bank Account class.
- Demonstration of deposit and withdrawal operations with updated balance.
- Well-commented code with a clear explanation.

Prompt:



Code:

```
14 # BankAccount class for basic banking operations
15 class BankAccount:
16     """A simple banking account class to manage deposits and withdrawals."""
17     def __init__(self, account_holder, initial_balance=0):
18         """
19         Initialize a bank account with account holder name and initial balance.
20
21         Args:
22             account_holder (str): Name of the account holder
23             initial_balance (float): Starting balance (default: 0)
24         """
25         self.account_holder = account_holder
26         self.balance = initial_balance
27
28     def deposit(self, amount):
29         """
30         Deposit money into the account.
31
32         Args:
33             amount (float): Amount to deposit (must be positive)
34         """
35         if amount > 0:
36             self.balance += amount
37             print(f"Deposited ${amount:.2f}. New balance: ${self.balance:.2f}")
38         else:
39             print("Deposit amount must be positive.")
40
41     def withdraw(self, amount):
42         """
43         Withdraw money from the account with balance verification.
44
45         Args:
46             amount (float): Amount to withdraw
47         """
48         if amount > self.balance:
49             print(f"Insufficient funds. Available balance: ${self.balance:.2f}")
50
51         if amount > self.balance:
52             print(f"Insufficient funds. Available balance: ${self.balance:.2f}")
53         elif amount > 0:
54             self.balance -= amount
55             print(f"Withdrew ${amount:.2f}. New balance: ${self.balance:.2f}")
56         else:
57             print("Withdrawal amount must be positive.")
58
59     def check_balance(self):
60         """Display current account balance."""
61         print(f"Account holder: {self.account_holder}")
62         print(f"Current balance: ${self.balance:.2f}")
63
64 # Example usage
65 account = BankAccount("Alice", 500)
66 account.check_balance()
67 account.deposit(200)
68 account.withdraw(100)
69 account.withdraw(1000) # Insufficient funds
70 account.check_balance()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

All approaches verified!

Account holder: Alice

Current balance: \$500.00

Deposited \$200.00. New balance: \$700.00

Withdrew \$100.00. New balance: \$600.00

Insufficient funds. Available balance: \$600.00

Insufficient funds. Available balance: \$600.00

Account holder: Alice

Current balance: \$600.00

PS C:\Users\abbas

Explanation:

This code defines a **BankAccount** class that simulates basic banking operations like depositing, withdrawing, and checking balance. The `__init__` method initializes the account holder's name and starting balance. The `deposit()` method adds money to the account only if the amount is positive, while the `withdraw()` method subtracts money after checking if there are enough funds and if the amount is valid. The `check_balance()` method displays the account holder's name and current balance. In the example usage, an account is created for "Alice" with an initial balance of 500, then deposits and withdrawals are performed, including a failed withdrawal due to insufficient funds, showing how the class manages transactions and updates the balance.